

Architecture Specification

Agentic Access to ATLAS DAQ/OKS Historical Configuration

Natural Language to OksQuery

Architecture Type:	Agentic configuration-query system
Primary Interface:	MCP Server
Implementation:	Python
Configuration Backend:	TDAQ config / oksconfig
Configuration Engine:	OKS
Query Language:	OksQuery
Revision Selection:	oksconfig connection parameter
Access Mode:	Strictly read-only
Primary Historical Store:	ATLAS OKS Git repository

Architecture Design Document

Version 2.0

Contents

1	Document Purpose	1
1.1	Executive Recommendation	1
2	Architectural Objectives	1
2.1	Natural-Language Access	1
2.2	Historical Configuration Access	1
2.3	Reuse of Existing TDAQ Infrastructure	2
2.4	Read-Only Operation	2
2.5	Cached Historical Sessions	3
2.6	Revision-Correct Querying	3
3	Architectural Principles	3
3.1	Principle 1 — Existing TDAQ/OKS Components Remain the Authority	3
3.2	Principle 2 — LLM as Semantic Layer, Not Infrastructure Layer	4
3.3	Principle 3 — Native Validation	4
3.4	Principle 4 — No Independent XML Interpretation	4
3.5	Principle 5 — Historical Revision Before Schema	5
3.6	Principle 6 — Controlled Fallbacks	5
4	System Context	5
4.1	External User	5
4.2	MCP Server	5
4.3	ATLAS OKS Git Repository	6
4.4	Run Number Database	6
4.5	TDAQ Configuration Stack	6
5	High-Level Architecture	6
6	Module Classification	7
6.1	New Application Components	7
6.2	Existing Components	8
6.3	External Dependencies and Excluded Backends	9
7	Detailed Module Architecture	9
7.1	Architectural Layering	9
8	Module 1 – User / Operator	10
8.1	Purpose	10
8.2	Input	10
8.3	Output	10
8.4	Responsibilities	11
9	Module 2 – MCP Server	11
9.1	Purpose	11
9.2	Input	11
9.3	Output	11
9.4	Responsibilities	11
9.5	Security Boundary	12

10 Module 3 – Intent Extractor	12
10.1 Purpose	12
10.2 Input	12
10.3 Output	13
10.4 Intent Fields	13
10.5 Important Boundary	13
10.6 Failure Conditions	13
11 Module 4 – Historical Configuration Resolver	14
11.1 Purpose	14
11.2 Input	14
11.3 Primary Resolution Mechanism	14
11.4 Why Git Tags Are Preferred	14
11.5 Git Tag Resolver	15
11.6 Git Tag Not Found	15
12 Module 5 – Run Number Database Fallback	15
12.1 Purpose	15
12.2 Input	15
12.3 Retrieved Information	15
12.4 Architectural Rule	16
12.5 Fallback Label	16
13 Module 6 – Revision / Version Validator	16
13.1 Purpose	16
13.2 Why Validation Is Required	16
13.3 Input	16
13.4 Validation	16
13.5 Output	17
13.6 Security Principle	17
14 Module 7 – TDAQ Configuration Access Layer	17
14.1 Purpose	17
14.2 Primary Technology	17
14.3 Responsibilities	18
15 Module 8 – libpyconfig	18
15.1 Purpose	18
15.2 Flow	18
15.3 Why It Is Reused	18
15.4 Important Security Consideration	19
16 Module 9 – config::Configuration	19
16.1 Purpose	19
16.2 Primary Path	19
16.3 Selected Backend Boundary	19
17 Module 10 – oksconfig	19
17.1 Purpose	19
17.2 Primary Flow	19
17.3 Responsibilities	20

18 Module 11 – OksConfiguration	20
18.1 Purpose	20
18.2 Flow	20
19 Module 12 – OksKernel	20
19.1 Purpose	20
19.2 Responsibilities	20
19.3 Historical Configuration	21
20 Module 13 – ATLAS OKS Git Repository	21
20.1 Purpose	21
20.2 Access	21
20.3 Read-Only Requirement	21
21 Module 14 – Historical Git Checkout	21
21.1 Purpose	21
21.2 Flow	22
21.3 Temporary Repository	22
22 Module 15 – Loaded In-Memory OKS Configuration	22
22.1 Purpose	22
22.2 Flow	22
23 Module 16 – Schema Retriever	22
23.1 Purpose	22
23.2 Existing APIs	22
23.3 Flow	23
23.4 Why Schema Retrieval Is Required	23
24 Module 17 – Schema Context Builder	23
24.1 Purpose	23
24.2 Input	24
24.3 Output	24
24.4 Conceptual Flow	24
25 Module 18 – Query Generation Context	24
25.1 Purpose	24
25.2 Architecture	24
26 Module 19 – LLM	25
26.1 Purpose	25
26.2 Strict Access Boundary	25
26.3 Required Flow	25
27 Module 20 – OksQuery Generator	25
27.1 Purpose	25
27.2 Input	26
27.3 Output	26
27.4 Important Architectural Rule	26
28 Module 21 – Native OksQuery	26
28.1 Purpose	26
28.2 Flow	26

29 Module 22 – OksQuery.good() Validator	26
29.1 Purpose	26
29.2 Validation	27
29.3 Decision	27
29.3.1 Valid Query	27
29.3.2 Invalid Query	27
29.4 Bounded Repair	27
30 Module 23 – Validation Diagnostic	27
30.1 Purpose	27
30.2 Flow	27
31 Module 24 – Bounded Query Repair	27
31.1 Purpose	27
31.2 Important Constraint	28
31.3 Failure	28
32 Module 25 – Query Executor	28
32.1 Purpose	28
32.2 Existing Execution API	28
32.3 Flow	28
33 Module 26 – Result Limiter	28
33.1 Purpose	28
34 Module 27 – Result Formatter / Serializer	29
34.1 Purpose	29
34.2 Flow	29
35 Module 28 – Answer Generator	29
35.1 Purpose	29
36 Module 29 – Central Error Handler	29
36.1 Purpose	29
37 Module 30 – MCP Response Builder	29
37.1 Purpose	29
38 Module 31 – Configuration Session Cache	29
38.1 Purpose	29
38.2 Architecture	30
38.3 Cache Policy	30
39 Module 32 – Query Pre-flight Validator	30
39.1 Purpose	30
39.2 Validation Boundary	30
39.3 Authoritative Validator	30
40 Module 33 – Provenance and Revision Listing Adapter	30
40.1 Purpose	30
40.2 Required Provenance	30

41 Module 34 – Security and Operational Boundary	31
41.1 Purpose	31
41.2 Allowed Path	31
41.3 Forbidden Paths	31
41.4 Authentication and Authorization Finding	31
41.5 Deployment Guardrails	31
42 Primary Data Flows	32
42.1 Flow A — Historical Resolution	32
42.2 Flow B — Query Generation	32
42.3 Flow C — Query Execution and Answer	32
43 Minimum Useful Scope	32
44 Architectural Separation of Responsibilities	33
45 Existing Versus New Components	33
45.1 Existing Components	33
45.2 New Application Components	34
46 Explicitly Excluded Components	35
47 Release-Specific Constraints and Risks	36
47.1 Operational and Historical-Data Constraints	36
47.2 Risk Register	36
48 Questions Requiring ATLAS/TDAQ Confirmation	36
49 Evidence Index	37
50 Core Architectural Principle	37

1 Document Purpose

This document specifies the proposed prototype architecture for **Agentic Access to ATLAS DAQ/OKS Historical Configuration**. It incorporates the latest repository investigation and supersedes the earlier generic backend design.

The system provides a natural-language interface through which a user can ask questions about the configuration of the ATLAS DAQ system at a particular historical run.

The central architectural objective is:

Natural Language → **Intent** → **Historical Run Resolution** → **Validated Revision** → **Existing TDAQ Configuration Stack** → **Existing OKS Kernel** → **Schema Context** → **OksQuery Generation** → **Native OksQuery Validation** → **Existing OKS Execution** → **Result Serialization** → **Natural Language Answer**

The architecture deliberately treats the existing TDAQ and OKS software as the configuration, historical-checkout, query-validation, and query-execution engine. The new system does not replace OKS, implement another configuration engine, parse the OKS XML files independently, or implement a second OksQuery parser. Its genuinely new code is limited to run resolution, schema selection and rendering, query generation, session caching, result serialization, provenance, and the MCP surface.

1.1 Executive Recommendation

Build a strictly read-only Python MCP server on the existing `config` binding, select `oksconfig`, and pass the historical revision through the per-session connection-string version parameter.

```
oksconfig:<config_name>&version=tag:r<run>@<partition>
```

This boundary already provides Python query execution, schema introspection, historical checkout, native schema-aware validation, and native execution. New code is required only above that boundary. The release does not provide a read-side run-to-revision resolver, and its WebDAQ HTTP interface cannot supply either a query or a revision.

2 Architectural Objectives

The prototype has the following objectives.

2.1 Natural-Language Access

The system shall allow a user to express a configuration question using natural language. For example, a user may ask a question conceptually equivalent to:

“Which object of class X was configured with property Y in run R ?”

The system converts the question into a structured intent and then into an OksQuery that can be evaluated against the historical configuration.

2.2 Historical Configuration Access

The system shall resolve a run number to the corresponding historical configuration revision and configuration entry file. The preferred mechanism is the existing run-tag convention:

```
r<run>@<partition>
```

For example:

```
r123456@ATLAS
```

The tag identifies the configuration revision associated with the run and partition. The resolved revision is passed per configuration session, not through process-global state:

```
oksconfig:<config_name>&version=tag:r<run>@<partition>
```

This connection-string form is preferred because it is scoped to one `Configuration` instance and is safe for concurrent sessions. If the tag is absent, the prototype queries the Run Number database for `CONFIGVERSION`, `CONFIGNAME`, and partition metadata. The environment variable `TDAQ_DB_VERSION` is retained only as a process-global, non-concurrent fallback.

2.3 Reuse of Existing TDAQ Infrastructure

The prototype shall reuse existing components wherever possible. Important existing components include:

- `libpyconfig`
- `config::Configuration`
- `oksconfig`
- `OksConfiguration`
- `OksKernel`
- `Configuration::get_class_info()`
- `Configuration::subclasses()`
- Python schema methods `classes()`, `attributes()`, and `relations()`
- `OksQuery`
- `OksQuery::good()`
- `OksClass::execute_query()`
- `conf.get_objs()`
- `Configuration.get_objs(class, query)`
- `OksRepositoryVersion` metadata
- Existing OKS Git integration

No custom replacement should be created for these capabilities.

2.4 Read-Only Operation

The prototype shall operate as a read-only service. The intended operation is:

User → Application → TDAQ/OKS read APIs → Historical Configuration

There shall be no application path that performs:

- configuration mutation,

- `commit()`,
- `create_obj()` or `destroy_obj()`,
- `create_db()`,
- `add_dal()`, `update_dal()`, or `destroy_dal()`,
- `add_include()` or `remove_include()`,
- `set_commit_credentials()`,
- Git push,
- configuration creation,
- configuration deletion,
- RDB writer operations.

The presence of write APIs in the Python binding does not change this architectural decision. Read-only behaviour must be enforced by the MCP tool surface, application design, deployment permissions, and read-only repository credentials.

2.5 Cached Historical Sessions

The prototype shall cache loaded configuration sessions by (revision, config name). The release implementation performs a full Git clone into a fresh temporary directory for each new kernel, so opening a new kernel for every request is operationally unacceptable. Cache entries remain read-only and expose provenance for the exact checked-out revision.

2.6 Revision-Correct Querying

A generated query must be validated against the actual schema loaded for the historical revision being queried. This is important because a class, attribute, relation, or type may change between configuration revisions. Therefore:

Resolve Revision → Load Revision → Retrieve Schema → Generate Query → Validate Query

rather than generating a query against a generic or current schema.

3 Architectural Principles

3.1 Principle 1 — Existing TDAQ/OKS Components Remain the Authority

The prototype does not become the source of truth for configuration. The authoritative configuration remains the historical OKS configuration accessed through the existing TDAQ configuration stack. The LLM is therefore not allowed to invent configuration facts. The LLM proposes a query; the existing OKS system determines the actual result.

3.2 Principle 2 — LLM as Semantic Layer, Not Infrastructure Layer

The LLM performs semantic tasks such as:

- understanding the user’s question,
- identifying entities and concepts,
- selecting relevant schema information,
- generating OksQuery syntax,
- repairing invalid generated queries,
- producing a natural-language explanation of returned results.

The LLM must not directly perform infrastructure operations. In particular, the LLM shall not directly access:

- Git,
- the filesystem,
- the shell,
- OKS,
- the Run Number database,
- TDAQ services.

All infrastructure access is performed by controlled application modules.

3.3 Principle 3 — Native Validation

The system shall not implement a custom OksQuery validator. The generated query is passed to the native:

```
OksQuery(class_name, query_text)
```

and validated through:

```
good()
```

This provides validation against the actual OKS query implementation and the loaded configuration schema.

3.4 Principle 4 — No Independent XML Interpretation

The application shall not independently parse the OKS XML files to reconstruct the configuration. The existing OKS kernel already handles:

- configuration loading,
- include resolution,
- schema loading,
- object loading,
- inheritance,
- Git-backed configuration revisions.

Reimplementing these mechanisms would introduce unnecessary duplication and potentially produce behaviour different from the actual TDAQ system.

3.5 Principle 5 — Historical Revision Before Schema

Schema retrieval occurs only after the historical configuration has been resolved and loaded. The schema is therefore tied to:

(revision, partition, class)

rather than being treated as a global static resource.

3.6 Principle 6 — Controlled Fallbacks

Fallback mechanisms shall not become hidden alternative primary paths. The primary historical-resolution path is:

Run Number + Partition $\rightarrow r\langle run \rangle @ \langle partition \rangle \rightarrow$ Git Revision

The derivable Git tag is the first lookup because run allocation records that tag at run start. The Run Number database is the authoritative fallback for `CONFIGVERSION` and `CONFIGNAME`. Neither path is guaranteed: tag creation is best-effort and the database version may be null. The service must report an unresolvable run honestly rather than substituting the current configuration.

The `roksconfig`, CORAL archive, `rdbconfig`, and live RDB paths are excluded from the prototype because their historical role is not established for this release.

4 System Context

4.1 External User

The user is the operator or researcher asking a historical configuration question. The user communicates with the system through the MCP interface. The user does not directly interact with:

- OKS,
- Git,
- the Run Number database,
- TDAQ configuration APIs.

4.2 MCP Server

The MCP server is the application-facing entry point. Its responsibility is to:

- receive a request,
- invoke the appropriate application workflow,
- return the final structured or natural-language response.

The MCP server is not itself the configuration engine. It orchestrates the application modules that perform the actual historical configuration access and query processing.

The implementation-specific MCP tool surface is:

- `resolve_run(run, partition);`

- `describe_schema(classes?);`
- `validate_query(class, query);`
- `run_query(class, query);`
- `list_revisions().`

4.3 ATLAS OKS Git Repository

The Git repository is the primary historical configuration source. It contains versioned OKS configuration data and schema. The application uses the repository in read-only mode. The prototype does not push changes to the repository.

4.4 Run Number Database

The Run Number database is a fallback source for resolving a run to configuration metadata. The release contains the write side of this mapping but no reusable read-side API for the MCP. New resolver code must therefore retrieve:

- configuration version,
- configuration name,
- partition name.

The Run Number database is not the first lookup, but it is authoritative when the derivable Git tag is missing. A null `CONFIGVERSION` means that the run cannot be resolved through this route.

4.5 TDAQ Configuration Stack

The existing TDAQ configuration stack provides the controlled interface between the application and OKS. The relevant logical stack is:



The Python binding already accepts a query string through `get_objs(class, query)`, and `oksconfig` already invokes the native `OksQuery` validator and executor. This stack is reused instead of recreated.

5 High-Level Architecture

The system consists of the following logical stages:

1. MCP request handling
2. Intent extraction
3. Historical configuration resolution
4. Revision validation
5. Configuration-session cache lookup

6. Historical configuration loading
7. Schema retrieval
8. Query-generation context construction
9. OksQuery generation
10. Lightweight pre-flight validation
11. Native schema-aware query validation
12. Bounded query repair
13. Query execution
14. Result limiting
15. Result formatting
16. Provenance assembly
17. Natural-language answer generation
18. MCP response

The primary data flow is illustrated in fig. 1.

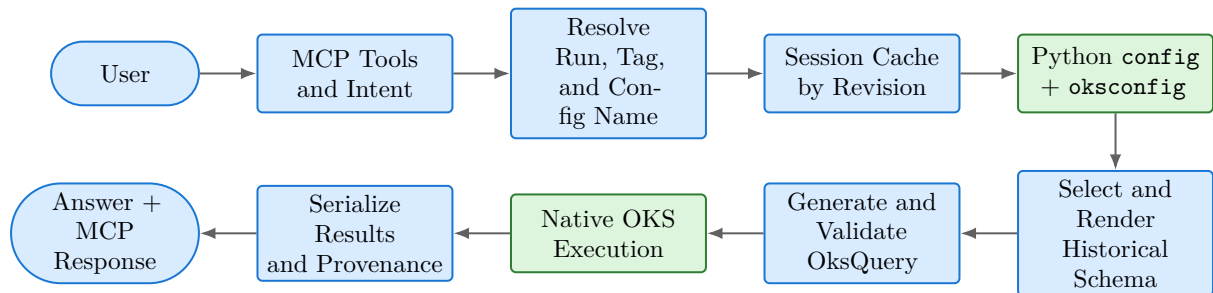


Figure 1: End-to-end architecture for historical configuration queries

An editable version of the main architecture diagram is available in [Whimsical](#).

6 Module Classification

Each component is classified according to whether it is newly implemented by the prototype or reused from the existing TDAQ/OKS ecosystem.

6.1 New Application Components

The prototype introduces the following logical modules:

- Intent Extractor
- Historical Configuration Resolver
- Run Tag Resolver
- Revision / Version Validator

- Configuration Session Cache
- Schema Context Builder
- Query Generation Context Builder
- LLM integration
- OksQuery Generator
- Query Pre-flight Validator
- Query Repair Controller
- Result Limiter
- Result Formatter / Serializer
- Provenance Builder
- Revision Listing Adapter
- Answer Generator
- Error Handler

These components form the new semantic and orchestration layer.

6.2 Existing Components

The following components are reused:

- `libpyconfig`
- `config::Configuration`
- `oksconfig`
- `OksConfiguration`
- `OksKernel`
- OKS Git integration
- `Configuration::get_class_info()`
- `Configuration::subclasses()`
- `Configuration.get_objs(class, query)`
- `OksQuery`
- `OksQuery::good()`
- `OksClass::execute_query()`
- `conf.get_objs()`
- OKS object APIs

The application must use these interfaces rather than replacing them.

6.3 External Dependencies and Excluded Backends

The prototype depends on the following external services or runtime conditions:

- Run Number database endpoint and read credentials
- read access to TDAQ_DB_REPOSITORY
- a configured TDAQ runtime environment
- `git` and the OKS checkout scripts on `PATH`
- backend libraries available to `dlopen`

The `roksconfig`/CORAL archive, `rdbconfig`/CORBA, and WebDAQ HTTP paths are explicitly excluded from the primary architecture. The HTTP endpoint exposes neither an `OksQuery` parameter nor a historical revision parameter in this release.

7 Detailed Module Architecture

The preceding sections define the system goals, boundaries, context, and high-level flow. The remainder of this document develops that architecture module by module, following the same processing order from the user-facing interface to historical configuration access, query execution, and the final response.

The architecture is designed around the principle that the prototype should add semantic intelligence around the existing TDAQ/OKS configuration infrastructure rather than replace any part of that infrastructure.

The overall processing principle is:

Natural Language → Intent → Historical Run Resolution → Validated Revision → Existing TDAQ Configuration Stack → Existing OKS Kernel → Schema Context → OksQuery Generation → Native OksQuery Validation → Existing OKS Execution → Result Serialization → Answer Generation → MCP Response

The application is strictly read-only. No module in the prototype is permitted to modify an OKS configuration, create a commit, push to a Git repository, or modify production configuration state.

7.1 Architectural Layering

The system can be divided into the following logical layers:

1. Client and protocol layer
2. Natural-language understanding layer
3. Historical configuration resolution layer
4. Configuration access layer
5. Schema understanding layer
6. Query generation layer
7. Native query validation layer
8. Query execution layer

9. Result processing layer
10. Natural-language answer layer
11. Error handling layer

The distinction between these layers is important because the LLM should not become responsible for infrastructure operations. The LLM performs semantic interpretation and query generation, while the application and existing TDAQ/OKS libraries perform configuration access, validation, and execution.

8 Module 1 – User / Operator

8.1 Purpose

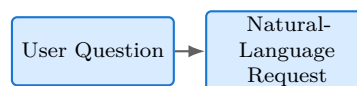
The User / Operator is the external source of a natural-language request. The user asks questions about historical ATLAS DAQ/OKS configuration. Example questions include:

- What configuration was used for run 123 456?
- Which processes were configured in partition P1 during run 123 456?
- What was the configuration of the XYZ component in run 123 456?
- Show the value of attribute A for all objects of class B in run 123 456.

The user does not need to know OksQuery syntax.

8.2 Input

The input is a natural-language question. Conceptually:



8.3 Output

The user-facing system eventually returns a natural-language answer containing structured provenance information. The final response should normally expose:

- interpretation of the question;
- run number;
- partition;
- resolved configuration revision;
- answer or result set;
- generated OksQuery;
- truncation information, if applicable.

8.4 Responsibilities

The User / Operator module has no responsibility for:

- Git operations;
- configuration loading;
- schema parsing;
- OksQuery validation;
- query execution;
- configuration modification.

It is simply the external consumer of the system.

9 Module 2 – MCP Server

9.1 Purpose

The MCP Server is the application-facing protocol boundary of the prototype. It exposes the historical configuration query capability to an MCP-compatible client. The MCP server should be considered the entry and exit point of the application rather than the component responsible for performing the actual TDAQ/OKS operations.

9.2 Input

The MCP server receives a request containing the user's question and, depending on the client protocol, associated request metadata. Conceptually:

MCP request fields:

- natural-language question;
- optional run number and partition;
- request metadata.

9.3 Output

The MCP server returns the final structured response. Conceptually:

MCP response fields:

- interpretation, run number, partition, and revision;
- generated query and structured results;
- natural-language answer and truncation metadata.

9.4 Responsibilities

The MCP server is responsible for:

1. receiving requests;
2. invoking the application pipeline;
3. returning structured responses;

4. exposing errors in a controlled format;
5. preventing direct access from the client to internal TDAQ components.

The minimum implementation-specific tool surface is:

resolve_run	Resolve a run and partition to a revision and configuration entry file.
describe_schema	Return the class list and selected class details, including one relationship hop.
validate_query	Perform pre-flight checks followed by native OksQuery validation.
run_query	Execute a validated read-only query and return capped JSON results.
list_revisions	Expose repository versions and provenance supported by the existing configuration stack.

9.5 Security Boundary

The MCP server forms part of the application's trust boundary. The following direct paths must not exist:

The MCP client must not directly access Git, the filesystem, OksKernel, configuration-mutation APIs, or the shell.

Instead:



10 Module 3 – Intent Extractor

10.1 Purpose

The Intent Extractor converts the natural-language question into a structured representation that can be processed deterministically by the rest of the system. This module is one of the new application-level components. Its primary purpose is to separate semantic interpretation from infrastructure resolution.

10.2 Input

The Intent Extractor receives:

- original user question;
- optional request metadata supplied by the MCP client.

Example:

"What processes were configured in partition P1 during run 123456?"

10.3 Output

The module produces a structured intent. A conceptual representation is:

```
{
  run_number: 123456,
  partition: "P1",
  concept: "processes",
  filters: [...],
  requested_fields: [...],
  operation: "retrieve"
}
```

The exact implementation representation may be JSON, Python objects, dataclasses, or another internal representation.

10.4 Intent Fields

The structured intent should contain, where applicable:

run_number	The requested historical run number.
partition	The ATLAS/TDAQ partition associated with the request.
concept	The conceptual entity the user is asking about.
filters	Constraints expressed by the user.
requested_fields	Attributes or properties requested by the user.
operation	The type of read operation requested.

10.5 Important Boundary

The Intent Extractor should extract the run number. It must **not** decide which Git revision corresponds to the run. For example:



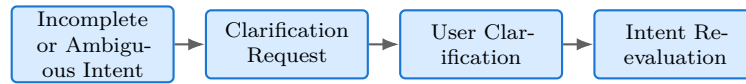
This separation prevents the LLM from inventing or guessing historical configuration revisions.

10.6 Failure Conditions

The Intent Extractor may be unable to produce a complete, reliable intent if:

- the question is ambiguous;
- no meaningful configuration-related intent can be extracted;
- the requested operation is unsupported;
- required information is missing.

In these cases, the system must not guess missing values, select a likely interpretation, or continue with assumed defaults. Instead, the Intent Extractor returns a clarification-required result through the MCP server. The response should briefly state what was understood, identify the ambiguous or missing information, and ask one focused follow-up question.



The user's clarification is combined with the original request and passed through intent extraction again. If the requested operation is outside the prototype's supported scope, the response should explain the limitation and invite the user to reformulate the question. The central Error Handler is used only when clarification cannot resolve the request or when a non-semantic system failure occurs.

11 Module 4 – Historical Configuration Resolver

11.1 Purpose

The Historical Configuration Resolver determines both the historical revision and configuration entry file associated with the requested run. This read-side lookup is new code: the release records the mapping at run start, but no reusable component reads it back for this purpose.

11.2 Input

The resolver receives:

- run number;
- partition.

For example:

```
run_number = 123456
partition = P1
```

11.3 Primary Resolution Mechanism

The primary resolution mechanism is the run-specific Git tag:

```
r<run>@<partition>
```

For example:

```
r123456@P1
```

The resolver therefore performs:



11.4 Why Git Tags Are Preferred

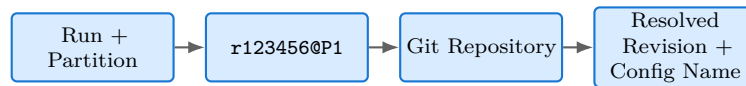
The repository investigation established that run allocation requests run-specific tags following this convention. The tag is derivable and is therefore the cheapest first lookup. Tag creation is best-effort on a background thread, however, so absence of a tag does not prove that the run has no historical mapping.

11.5 Git Tag Resolver

The Historical Configuration Resolver delegates the primary lookup to a Git Tag Resolver. The Git Tag Resolver:

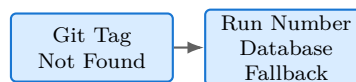
1. receives a run number;
2. receives a partition;
3. constructs the expected tag identifier;
4. validates the identifier;
5. queries the repository;
6. returns the resolved revision and configuration name.

Conceptually:



11.6 Git Tag Not Found

If the expected tag cannot be found, the system must not simply guess another revision. Instead, it activates the fallback path.



12 Module 5 – Run Number Database Fallback

12.1 Purpose

The Run Number Database provides the authoritative fallback when the expected Git run tag cannot be found. The database write side exists in the release, but the MCP must implement the read-side lookup and handle deployment-specific endpoint and credential configuration.

12.2 Input

The fallback receives:

- run number;
- partition where required.

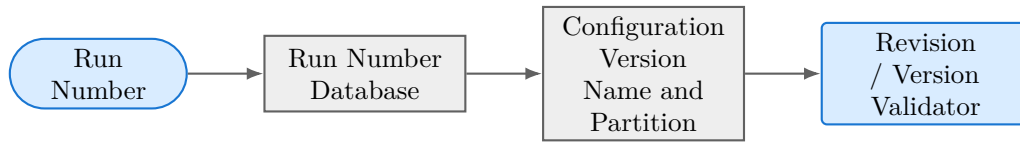
12.3 Retrieved Information

The database may provide:

- CONFIGVERSION;
- CONFIGNAME;
- PARTITIONNAME;

- historical configuration references.

The architecture should represent the fallback as:



12.4 Architectural Rule

The Run Number Database is not the historical configuration store. Its role is to recover the recorded mapping when the preferred tag lookup fails. If `CONFIGVERSION` is null, the resolver must return an explicit unresolvable-run result.

12.5 Fallback Label

In the architecture diagram this path should be labelled `FALLBACK -- Run Number DB` and represented using a dashed connection.

13 Module 6 – Revision / Version Validator

13.1 Purpose

The Revision / Version Validator is a security-critical boundary between historical resolution and configuration loading. Its purpose is to ensure that a revision identifier is valid before it is supplied to the TDAQ/OKS configuration-loading infrastructure.

13.2 Why Validation Is Required

The repository investigation found that configuration operations ultimately reach shell commands through calls involving `system()`. Consequently, a revision identifier must never be treated as arbitrary untrusted text. The LLM must never be allowed to generate an unrestricted revision string that is subsequently passed to configuration infrastructure.

13.3 Input

The validator receives a resolved version identifier from:

- Git Tag Resolver; or
- Run Number Database fallback.

13.4 Validation

The validator should enforce a strict grammar appropriate for each type of identifier.

For Git hashes, the prototype can use a restricted form such as:

```
^[0-9a-f]{7,40}$
```

For run tags, the expected structure is:

```
^r[0-9]+@[A-Za-z0-9_-]+$
```

The application then constructs the supported version form itself:

```
tag:r<run>@<partition>
```

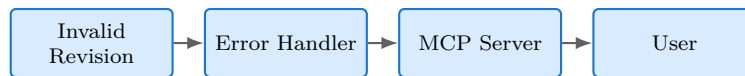
The exact accepted grammar should follow the production conventions confirmed with the TDAQ experts.

13.5 Output

If valid:

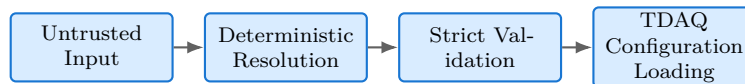


If invalid:



13.6 Security Principle

No generated natural-language content should be passed directly into a connection-string version parameter or TDAQ_DB_VERSION. The required flow is:



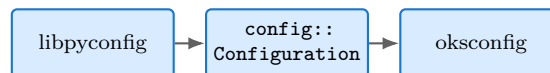
14 Module 7 – TDAQ Configuration Access Layer

14.1 Purpose

The TDAQ Configuration Access Layer is the application's controlled interface to the existing TDAQ configuration subsystem. This layer prevents higher-level application logic from depending on low-level implementation details.

14.2 Primary Technology

The prototype uses:



The application opens a historical session with a connection string of the form:

```
Configuration("oksconfig:<config_name>&version=tag:r<run>@<partition>")
```

The application should reuse these existing components rather than implementing an alternative configuration engine.

14.3 Responsibilities

The access layer is responsible for:

- selecting the appropriate configuration backend;
- loading the historical configuration;
- exposing configuration APIs to the application;
- providing access to schema information;
- providing read-only query execution.

It should not implement:

- XML parsing;
- Git checkout logic;
- OksQuery parsing;
- OksQuery validation;
- custom configuration persistence.

15 Module 8 – libpyconfig

15.1 Purpose

`libpyconfig` provides the Python binding through which the Python-based prototype communicates with the TDAQ configuration subsystem. This is the preferred application integration point.

The binding already exposes query execution through `get_objs(class, query)` as well as schema methods for classes, attributes, relations, subclasses, and superclasses. No new Python binding is required for the prototype.

15.2 Flow

The primary path is:



15.3 Why It Is Reused

The repository investigation confirmed that Python bindings already exist. Reimplementing the configuration interface would duplicate existing TDAQ functionality and introduce unnecessary compatibility risks.

15.4 Important Security Consideration

The Python binding exposes both read and write operations. Importing the `libpyconfig` module therefore does not guarantee read-only behaviour. The prototype must enforce read-only operation through:

1. deployment permissions;
2. application-level API discipline;
3. omission of all mutation operations from the prototype;
4. repository permissions that do not allow pushes.

The architecture therefore treats `libpyconfig` as an existing technology component but constrains its use to the read path.

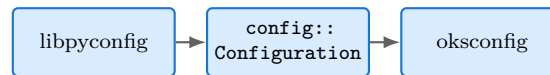
16 Module 9 – `config::Configuration`

16.1 Purpose

`config::Configuration` is the technology-neutral configuration interface used by the application. It provides the abstraction between the Python application and the specific configuration backend.

16.2 Primary Path

The prototype uses:



The important architectural property is that backend selection happens through the existing configuration plugin mechanism.

16.3 Selected Backend Boundary

The underlying plugin mechanism remains backend-independent, but this prototype deliberately selects `oksconfig`. Supporting other backends is not an application requirement and no provider abstraction layer is introduced.

17 Module 10 – `oksconfig`

17.1 Purpose

`oksconfig` is the primary backend selected for the first prototype. It connects the generic configuration interface to the OKS kernel and historical OKS repository.

17.2 Primary Flow



17.3 Responsibilities

The backend is responsible for:

- loading the requested configuration;
- interacting with the OKS kernel;
- resolving repository data;
- providing configuration objects;
- forwarding OksQuery operations.

It also parses the per-session `version=` connection parameter, constructs the native `OksQuery`, validates it with `good()`, and executes the same parsed query against the loaded class.

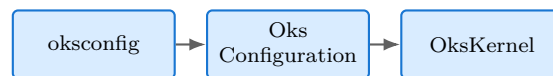
The prototype does not reimplement these functions.

18 Module 11 – OksConfiguration

18.1 Purpose

`OksConfiguration` is the existing OKS configuration implementation used by the `oksconfig` backend. It connects the generic configuration interface to the underlying `OksKernel`.

18.2 Flow



The application should interact with this functionality through the existing configuration API rather than creating an alternative OKS configuration implementation.

In the release implementation, `OksConfiguration::get()` creates the native query object, performs exact schema-aware validation, and then invokes `execute_query()`. Validation and execution therefore cannot drift between two different parsers.

19 Module 12 – OksKernel

19.1 Purpose

`OksKernel` is the existing OKS engine responsible for loading the configuration into memory and maintaining the in-memory classes and objects. It is one of the most important existing components in the architecture.

19.2 Responsibilities

The kernel handles functionality such as:

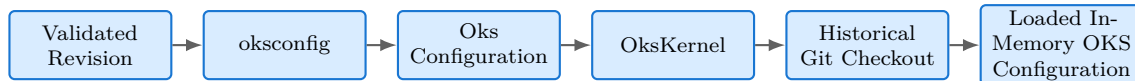
- loading OKS configuration;
- repository interaction;
- Git-based revision handling;
- include resolution;

- class loading;
- object loading;
- maintaining the in-memory configuration model.

The kernel supports historical selection through the connection-string version passed by `oksconfig`. It can also read `TDAQ_DB_VERSION`, but that environment variable is process-wide and must not be used for concurrent requests.

19.3 Historical Configuration

The prototype does not directly parse historical XML files. Instead:



This is an important architectural principle. The prototype delegates historical repository handling to the existing OKS/TDAQ implementation.

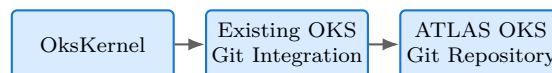
20 Module 13 – ATLAS OKS Git Repository

20.1 Purpose

The ATLAS OKS Git Repository is the external source of versioned historical configuration. It stores the versioned configuration data used by the OKS system.

20.2 Access

The repository is accessed through the existing TDAQ/OKS Git integration. The application should not implement its own Git checkout mechanism. The desired architecture is:



The actual repository endpoint used by the deployment must be confirmed with the ATLAS/TDAQ experts.

20.3 Read-Only Requirement

The prototype only requires repository read access. There must be no application path such as:

The application must not commit to Git, push repository changes, or modify configuration state.

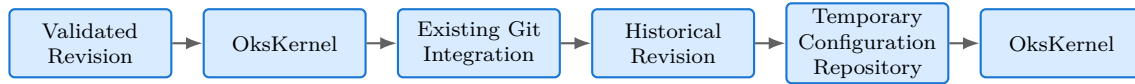
The prototype should operate using an identity that cannot modify the production repository.

21 Module 14 – Historical Git Checkout

21.1 Purpose

The Historical Git Checkout is performed by the existing TDAQ/OKS infrastructure. The application requests a specific historical revision, and the existing OKS infrastructure obtains the corresponding configuration.

21.2 Flow



The application must not introduce a custom Git implementation.

21.3 Temporary Repository

The release checkout script performs a full `git clone` for each new kernel and creates a fresh temporary directory. The prototype must therefore reuse loaded sessions through an application cache keyed by revision and configuration name. Temporary repositories are released when their cache entries are evicted or the service shuts down.

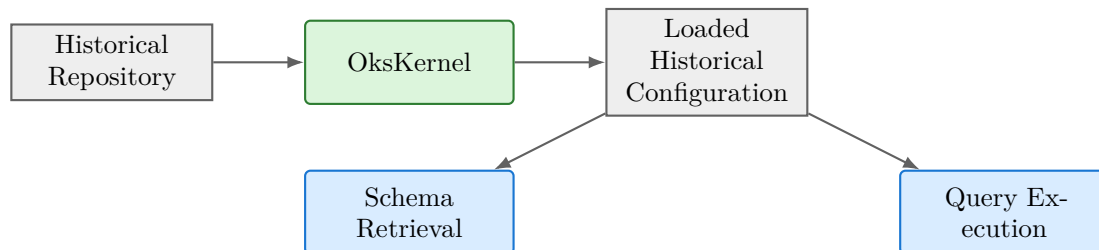
22 Module 15 – Loaded In-Memory OKS Configuration

22.1 Purpose

Once the historical configuration has been loaded, the OksKernel contains the classes and objects corresponding to that historical revision. This becomes the source for both:

1. schema retrieval;
2. query execution.

22.2 Flow



The schema and data are therefore tied to the same historical revision. This is important for correctness because the query generator must not generate a query against a schema from a different revision.

23 Module 16 – Schema Retriever

23.1 Purpose

The Schema Retriever obtains the schema information required to understand the historical configuration. The schema must be retrieved from the same loaded historical configuration that will eventually execute the generated query.

23.2 Existing APIs

The prototype reuses existing configuration APIs including:

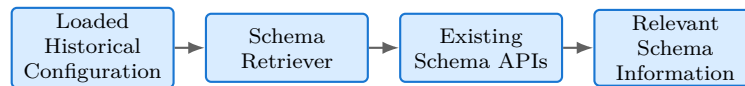
- `classes()`;

- `attributes(class, all=True);`
- `relations(class, all=True);`
- `subclasses()` and superclass information;
- `Configuration::get_class_info()` where appropriate.

Additional existing class information APIs can expose:

- classes;
- attributes;
- attribute types;
- relations;
- relation targets;
- inheritance information.

23.3 Flow



23.4 Why Schema Retrieval Is Required

The LLM cannot reliably generate an OksQuery merely from the natural language question. It must know the actual schema of the historical configuration. For example, if the user asks:

"Find all processes with state X"

the generator needs to know:

- which class represents a process;
- which attribute represents state;
- the type of the state attribute;
- the exact attribute identifier;
- relevant inheritance relationships.

24 Module 17 – Schema Context Builder

24.1 Purpose

The Schema Context Builder converts retrieved schema information into a compact context suitable for query generation. The prototype first measures the production schema size. Its default selection is the full class list plus detailed metadata for selected classes and one relationship hop; no vector or embedding index is introduced without evidence that simple schema context is insufficient.

24.2 Input

The module receives:

- structured intent;
- available schema information.

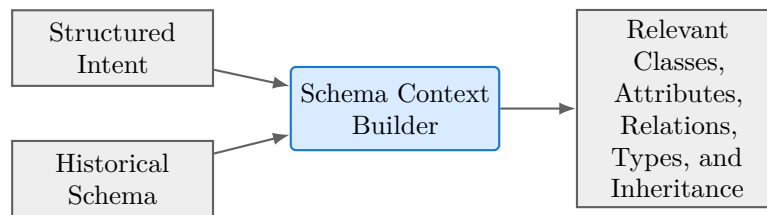
24.3 Output

It produces:

Relevant Schema Context

The context should contain only schema information relevant to the current question.

24.4 Conceptual Flow



This reduces unnecessary LLM context and reduces the probability of selecting unrelated classes.

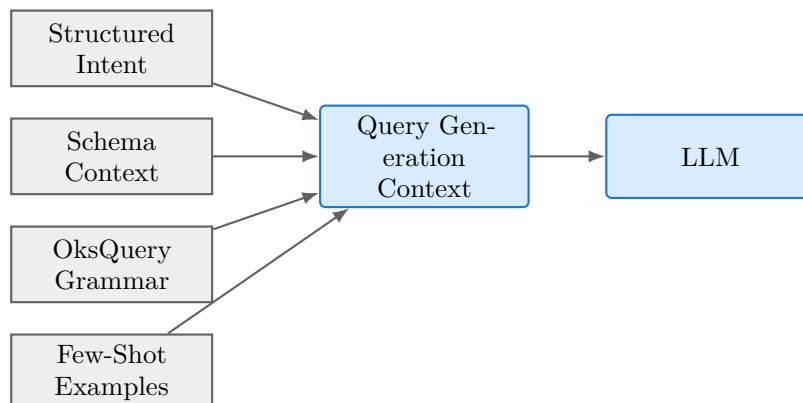
25 Module 18 – Query Generation Context

25.1 Purpose

The Query Generation Context is the complete controlled context supplied to the LLM for OksQuery generation. It combines four major inputs:

1. structured intent;
2. relevant schema context;
3. OksQuery grammar;
4. few-shot examples.

25.2 Architecture



The context builder is responsible for ensuring that the model receives the information necessary to construct a valid query without receiving unnecessary infrastructure details.

26 Module 19 – LLM

26.1 Purpose

The LLM provides semantic reasoning for:

- interpreting user intent;
- mapping natural-language concepts to schema entities;
- generating OksQuery expressions;
- generating natural-language answers from structured results.

The LLM is an application component, not an OKS replacement.

26.2 Strict Access Boundary

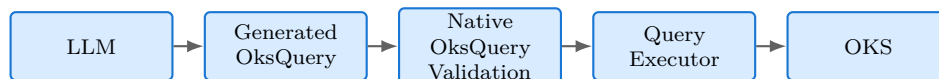
The LLM must not directly access:

- Git;
- filesystem;
- shell;
- OksKernel;
- Run Number Database;
- TDAQ infrastructure.

The application remains the controlled intermediary.

26.3 Required Flow

The LLM must follow:



There must not be:

The LLM must not directly access the shell, Git, the filesystem, or OKS.

27 Module 20 – OksQuery Generator

27.1 Purpose

The OksQuery Generator converts the structured query-generation context into an OksQuery request. This is one of the primary new components of the prototype.

27.2 Input

The generator receives:

- structured intent;
- relevant schema context;
- OksQuery grammar;
- few-shot examples.

27.3 Output

The generator should produce:

```
(class_name, oksquery_text)
```

For example:

```
class_name:
SomeClass

query:
("attribute" "value" =)
```

The exact query must be generated according to the actual schema and OksQuery grammar.

For the minimum prototype, generated expressions are restricted to the core grammar form (`this|all ( a v op )`). The separate `path-to/QueryPath` grammar is out of scope.

27.4 Important Architectural Rule

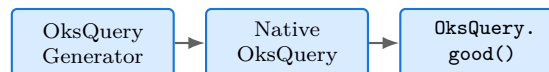
The generator does not execute the query. It only proposes a query. The proposed query must pass through the existing native OksQuery validation mechanism before execution.

28 Module 21 – Native OksQuery

28.1 Purpose

The generated query is passed to the existing native OksQuery implementation. The prototype must not implement another query engine.

28.2 Flow



The native implementation becomes the authoritative mechanism for determining whether the generated query is valid.

29 Module 22 – OksQuery.good() Validator

29.1 Purpose

`OksQuery.good()` is reused as the authoritative query validation mechanism. A lightweight application pre-flight may reject unknown classes or attributes earlier, but it must not become a second parser.

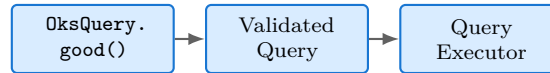
29.2 Validation

The existing validation mechanism checks the generated expression against the actual OKS schema and query rules. The validation therefore occurs against the same historical configuration that will execute the query. The same parsed query object is then supplied to native execution.

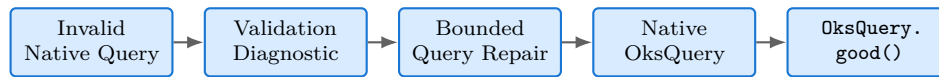
29.3 Decision

The architecture contains two branches.

29.3.1 Valid Query



29.3.2 Invalid Query



29.4 Bounded Repair

The repair mechanism should be bounded to approximately two or three attempts. The purpose of bounded repair is to correct deterministic query errors without allowing an uncontrolled model loop. Conceptually:

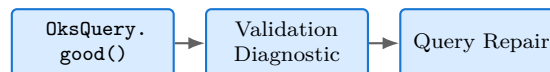


30 Module 23 – Validation Diagnostic

30.1 Purpose

The Validation Diagnostic contains information explaining why the generated query failed validation. The diagnostic is provided to the bounded repair process.

30.2 Flow



The diagnostic should not be treated as a substitute for the native validator. The native validator remains authoritative.

31 Module 24 – Bounded Query Repair

31.1 Purpose

The Bounded Query Repair module attempts to correct an invalid generated query. It receives:

- original structured intent;
- relevant schema context;

- generated query;
- validation diagnostic.

It then requests a corrected query from the LLM.

31.2 Important Constraint

The repair loop must remain bounded. A suitable prototype limit is:

$$N_{\max} = 3$$

where $N_{\max}$ represents the maximum number of validation attempts.

31.3 Failure

If the query remains invalid after the retry limit:



32 Module 25 – Query Executor

32.1 Purpose

The Query Executor executes a validated OksQuery against the loaded historical configuration. It reuses the existing OKS execution mechanism.

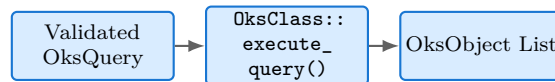
32.2 Existing Execution API

The Python-facing architecture uses:

```
Configuration.get_objs(class_name, query_text)
```

This existing binding reaches `config::Configuration::get()`, `OksConfiguration::get()`, and finally the read-only `OksClass::execute_query()` walk. The executor returns matching `ConfigObject` instances to the result-processing pipeline.

32.3 Flow



33 Module 26 – Result Limiter

33.1 Purpose

The Result Limiter applies deterministic bounds before results are serialized or sent to the LLM. It records whether the returned result set was truncated and, where available, the total number of matches.

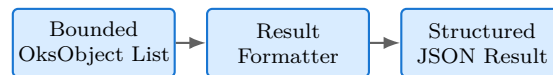
34 Module 27 – Result Formatter / Serializer

34.1 Purpose

The Result Formatter converts native OKS objects into a stable, JSON-compatible representation containing requested attributes, identifiers, class information, and historical provenance.

Serialization iterates the schema-defined attributes and relationships of each returned class. This is new application code, following the existing JSON-assembly pattern used by `webis_server`, but it does not reuse the WebDAQ HTTP query path.

34.2 Flow



35 Module 28 – Answer Generator

35.1 Purpose

The Answer Generator uses the structured result and provenance metadata to produce the final natural-language response. It must not add configuration facts that are absent from the executed query results.

36 Module 29 – Central Error Handler

36.1 Purpose

The Central Error Handler converts failures from every stage into controlled, structured errors. It distinguishes invalid input, unresolved revisions, configuration-loading failures, invalid queries, retry exhaustion, and execution failures.

37 Module 30 – MCP Response Builder

37.1 Purpose

The MCP Response Builder assembles the interpretation, run and partition, resolved Git SHA, configuration name, class, generated query, bounded results, truncation metadata, answer, and any structured error information for return to the client. These provenance fields are always returned so that a human can detect semantically wrong but syntactically valid queries.

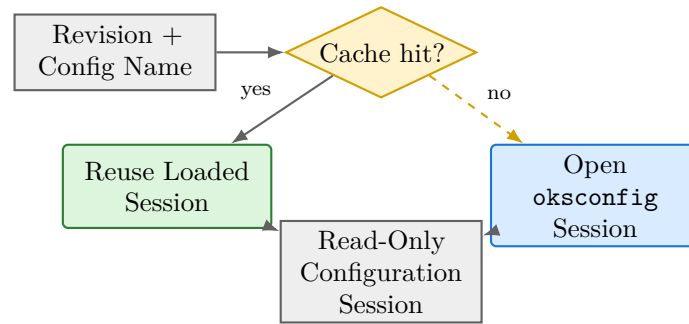
38 Module 31 – Configuration Session Cache

38.1 Purpose

The Configuration Session Cache prevents a full repository clone and kernel construction for every MCP request. It stores read-only `Configuration` sessions keyed by:

(resolved revision, configuration name).

38.2 Architecture



38.3 Cache Policy

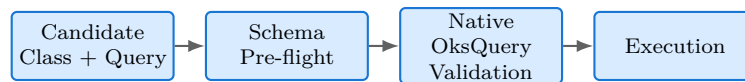
Entries are bounded by count or memory, released on eviction, and never shared across different revision/configuration keys. The connection string carries the version per session, so cached sessions do not rely on the process-global `TDAQ_DB_VERSION` variable.

39 Module 32 – Query Pre-flight Validator

39.1 Purpose

The Query Pre-flight Validator provides fast, readable feedback before the authoritative native parser is invoked. It checks that the requested class exists and that directly referenced attributes are present in the historical schema.

39.2 Validation Boundary



39.3 Authoritative Validator

Pre-flight checks are deliberately incomplete. The native `OksQuery` constructor and `good()` method remain the only authority for grammar, operand order, types, inheritance, and exact schema-aware validity. Native diagnostics are returned to the bounded repair loop.

40 Module 33 – Provenance and Revision Listing Adapter

40.1 Purpose

The Provenance and Revision Listing Adapter exposes existing repository-version metadata and configuration version APIs. The adapter implements the MCP operation:

```
list_revisions()
```

It also builds the audit record returned with every answer.

40.2 Required Provenance

- run number and partition;
- resolved Git SHA and original tag or database version;

- configuration name;
- queried class and exact OksQuery text;
- result count, cap, and truncation status.



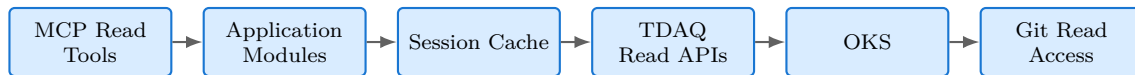
41 Module 34 – Security and Operational Boundary

41.1 Purpose

The entire prototype is designed as a read-only system. This is an architectural property enforced at the MCP surface, application API, repository identity, and deployment boundary.

41.2 Allowed Path

The allowed trust flow is:



41.3 Forbidden Paths

The architecture must not contain:

Forbidden operations include MCP-triggered commits, pushes, or configuration mutation; direct LLM access to infrastructure; and any application path that modifies production configuration.

The same Python object exposes mutation methods, so read-only behavior is not supplied by the library. The service must never expose or call the mutation surface listed in section 2.4.

41.4 Authentication and Authorization Finding

The release contains JWT, PAM, and AccessManager checks on write and repository-validation paths. No equivalent checks were found in the query execution path. This does not establish unrestricted read access; the architecture assumes that repository and deployment permissions control reads and records this as an expert-confirmation item.

41.5 Deployment Guardrails

- Set `OKS_GIT_DEBUG=no`. Otherwise the checkout script writes environment and process information into `.git/oks_proc_info`, which can expose service secrets.
- Ensure `git` and the `oks-*.sh` scripts are on `PATH`.
- Ensure `liboksconfig.so` and its dependencies are available to the runtime loader.
- Use a read-only repository identity and no write credentials.
- Cache sessions because checkout performs a full clone for each newly constructed kernel.

42 Primary Data Flows

The architecture can also be understood as three major data flows.

42.1 Flow A — Historical Resolution

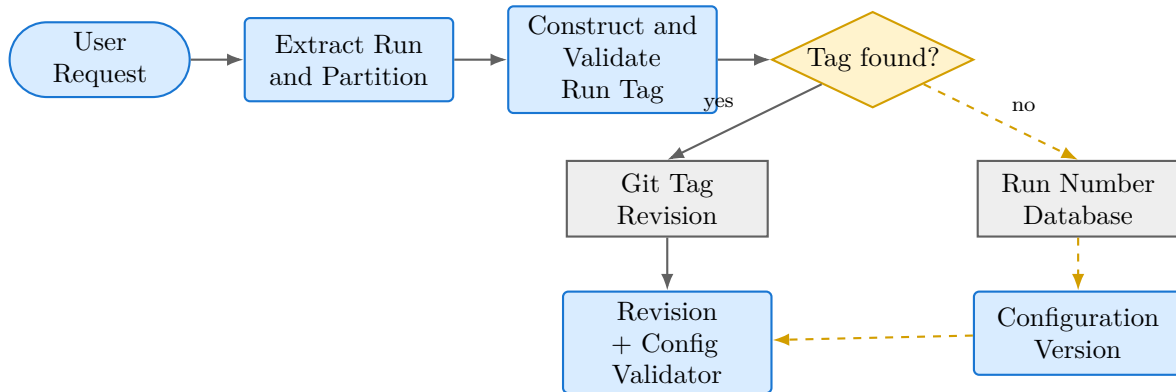


Figure 2: Run resolution to a validated revision and configuration name

42.2 Flow B — Query Generation

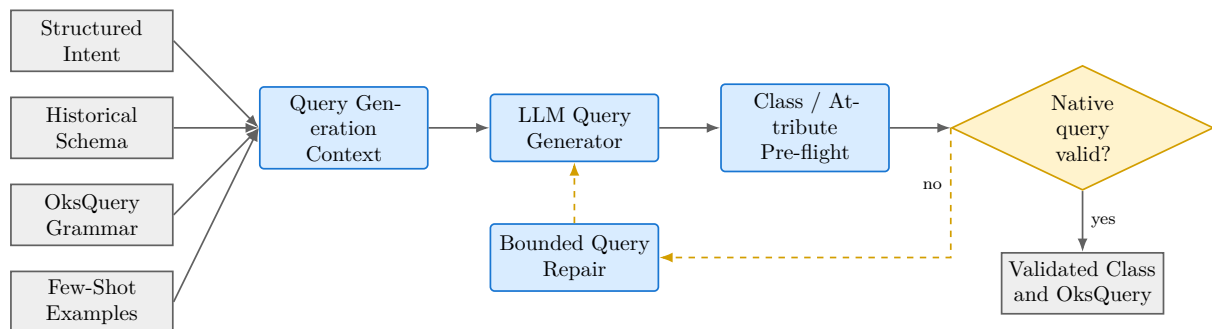


Figure 3: Schema-grounded query generation and bounded validation loop

42.3 Flow C — Query Execution and Answer

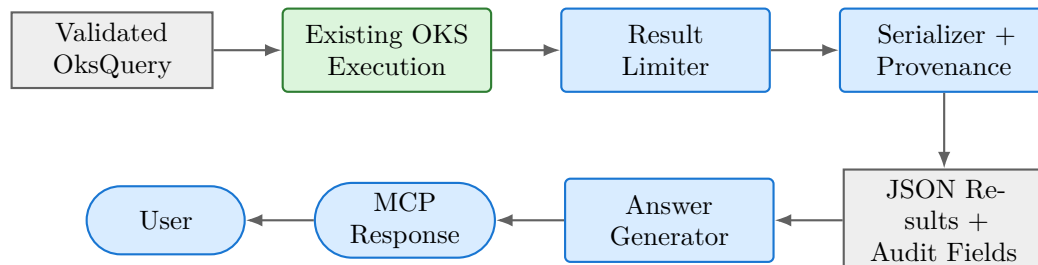


Figure 4: Validated query execution and response generation

43 Minimum Useful Scope

The minimum useful prototype is the following end-to-end read path:

1. `resolve_run(run, partition)` returns a validated revision and configuration name, using the derivable run tag first and the Run Number database as fallback.
2. `open(revision, config_name)` constructs a per-session `oksconfig` connection string and uses the session cache.
3. `describe_schema(classes?)` returns the class list and detailed metadata for selected classes plus one relationship hop.
4. `validate_query(class, query)` performs pre-flight checks and then invokes the real native validator.
5. `run_query(class, query)` executes the existing Python query path and serializes a capped result set.
6. The natural-language front end generates a class/query pair, performs bounded repair, and returns an answer with run, SHA, configuration name, class, query text, and truncation metadata.

The selected integration boundary is the existing `config` Python binding with the `oksconfig` backend and revision supplied through the `version=` connection parameter.

The WebDAQ / `webis_server` HTTP route is not a viable alternative for this scope because its request grammar exposes neither a query parameter nor a historical revision parameter.

44 Architectural Separation of Responsibilities

The most important separation in the system is between semantic intelligence and infrastructure execution.

This separation prevents the LLM from becoming a privileged infrastructure operator.

45 Existing Versus New Components

The prototype intentionally maximizes reuse.

45.1 Existing Components

The following components are reused:

- `libpyconfig`;
- `config::Configuration`;
- `oksconfig`;
- `OksConfiguration`;
- `OksKernel`;
- existing Git integration;
- existing configuration schema APIs;
- `Configuration.get_objs(class, query)`;
- `OksQuery`;

Area	Responsible Component
Natural-language interpretation	Intent Extractor / LLM
Run-to-revision mapping	Historical Configuration Resolver
Revision security validation	Revision / Version Validator
Session reuse and clone control	Configuration Session Cache
Configuration loading	Existing TDAQ configuration stack
Git checkout	Existing OKS Git integration
Schema retrieval	Existing Configuration APIs
Semantic query generation	LLM / OksQuery Generator
Early class/attribute checks	Query Pre-flight Validator
Query syntax/schema validation	Native <code>OksQuery.good()</code>
Query execution	Existing <code>OksClass::execute_query()</code>
Result serialization	Result Formatter / Serializer
Revision and audit metadata	Provenance Builder
Natural-language response	Answer Generator / LLM
Protocol delivery	MCP Server
Error propagation	Central Error Handler

Table 1: Responsibility separation

- `OksQuery.good()`;
- `OksClass::execute_query()`;
- existing typed configuration object accessors;
- `OksRepositoryVersion` metadata.

45.2 New Application Components

The prototype needs to implement:

- Intent Extractor;
- Historical Configuration Resolver glue;
- Revision / Version Validator;
- Configuration Session Cache;
- Schema Context Builder;
- Query Generation Context;
- OksQuery Generator;
- Query Pre-flight Validator;
- Bounded Query Repair;

- Result Limiter;
- Result Formatter / Serializer;
- Provenance and Revision Listing Adapter;
- Answer Generator;
- MCP Server shell;
- Central Error Handler.

The architecture should avoid creating replacements for existing TDAQ/OKS components.

46 Explicitly Excluded Components

The following functionality is outside the prototype scope:

- configuration mutation;
- `commit()`;
- `abort()`;
- `set_config_version()`;
- RDB writer functionality;
- `roksconfig` / CORAL archive integration;
- `rdbconfig` / live-partition access;
- WebDAQ / `webis_server` query extensions;
- Git-provider API clients or provider abstractions;
- new pybind11 or SWIG bindings;
- custom HTTP adapter;
- predictive maintenance;
- automatic configuration modification;
- custom XML parser;
- custom Git checkout implementation;
- custom OksQuery parser;
- custom OksQuery validator;
- custom configuration engine;
- semantic/vector retrieval;
- `path-to` / `QueryPath` queries;
- multi-partition or cross-revision joins.

These exclusions are deliberate. They prevent the six-week prototype from expanding into a replacement for the existing TDAQ/OKS infrastructure.

47 Release-Specific Constraints and Risks

47.1 Operational and Historical-Data Constraints

- Each newly constructed kernel performs a full Git clone into a temporary directory; the session cache is therefore required.
- `git`, the `oks-*.sh` scripts, and loadable backend libraries must be available in the service runtime.
- Set `OKS_GIT_DEBUG=no`; otherwise checkout records the process environment and process tree under `.git/oks_proc_info`.
- A run can have a missing tag and a null `CONFIGVERSION`; such a run is reported as unresolvable.
- Schema and data must always come from the same cached historical session because schemas can differ across revisions.
- No read-path authentication or authorization call was identified in the investigated query path. Repository and deployment access controls remain an unresolved external dependency.

47.2 Risk Register

Risk	Severity	Mitigation
Run-to-revision lookup unavailable	High	Try the derivable tag, confirm Run Number database access early, and return an explicit unresolved result.
Missing tag or null database version	High	Preserve both lookup routes; never substitute the current configuration.
No repository read access	High	Treat read-only clone access as a blocking deployment dependency.
Full clone latency	Medium	Cache by revision and configuration name; bound cache size and release temporary repositories on eviction.
Semantically wrong but valid query	High	Return run, SHA, config name, class, exact query text, result count, and truncation status.
Query grammar or operand-order errors	Medium	Use schema examples, pre-flight checks, native validation, and at most three repair attempts.
Environment leakage during checkout	Medium	Set <code>OKS_GIT_DEBUG=no</code> and avoid secrets in inherited environment variables where possible.
Runtime environment difficult to provision	Medium	Confirm the service host, TDAQ setup, <code>PATH</code> , and dynamic-library configuration early.

Table 2: Architecture risk register

48 Questions Requiring ATLAS/TDAQ Confirmation

1. Should run-to-revision resolution prefer the `r<run>@<partition>` tag or the Run Number database in production, and how reliable are the tags?
2. What is the production value of `TDAQ_DB_REPOSITORY`, and can the service receive read-only clone access?
3. Is the `config` Python binding and its `get_objs(class, query)` signature supported for this service use case?
4. How should the service provision the TDAQ runtime environment?

5. Are read operations controlled beyond Git and host permissions?
6. Is the CORAL historical OKS archive still operational or supported?
7. How large is a representative ATLAS schema, and how frequently does it change across revisions?
8. Is a separate MCP service preferred to any future extension of `webis_server`?

49 Evidence Index

The following source locations record the evidence used for the updated architecture.

- `config/python/config/Configuration.py`: constructor and connection string (52–68), `get_objs` query parameter (117), schema methods (137–211), and mutation surface (213–590).
- `config/src/python/config.cpp`: Python `get_objs` binding (161–170) and exception translation.
- `config/config/Configuration.h`: query-bearing `get()` (698), commit credentials (1203–1217), and revision listing (1283).
- `config/config/Schema.h`: existing schema meta-model (52–163).
- `config/src/Configuration.cpp`: backend selection and `liboksconfig.so` plugin loading (127–147).
- `oksconfig/src/OksConfiguration.cpp`: per-connection revision handling (150, 191–202), query construction, validation, and execution (693–729).
- `oks/src/query.cpp`: core grammar (127–137), parser and schema checks (182–425), and read-only execution (431–535).
- `oks/src/kernel.cpp`: version parsing (757–773), environment fallback (930–958), checkout (5937–5997), token verification (6167), and PAM validation (6736–6759).
- `oks/oks/kernel.h`: `OksRepositoryVersion` metadata (516–531), document loading, and includes.
- `oks/scripts/oks-checkout.sh`: full clone (155), checkout (172–206), debug environment capture (214–223), and version output (233).
- `rn/src/lib.cpp`: run tag creation (104), conditional version capture and null case (148, 252), `CONFIGVERSION` (255), `CONFIGNAME` (274), and best-effort background tagging (317).
- `webis_server/src/oks_handler.cpp`: HTTP URL grammar without query/revision parameters and JSON serialization pattern (284, 292–303).
- `daq_tokens/README.md` and `oks/bin/oks_validate_repository.cpp`: JWT API and `AccessManager` authorization on write/repository-validation paths.

50 Core Architectural Principle

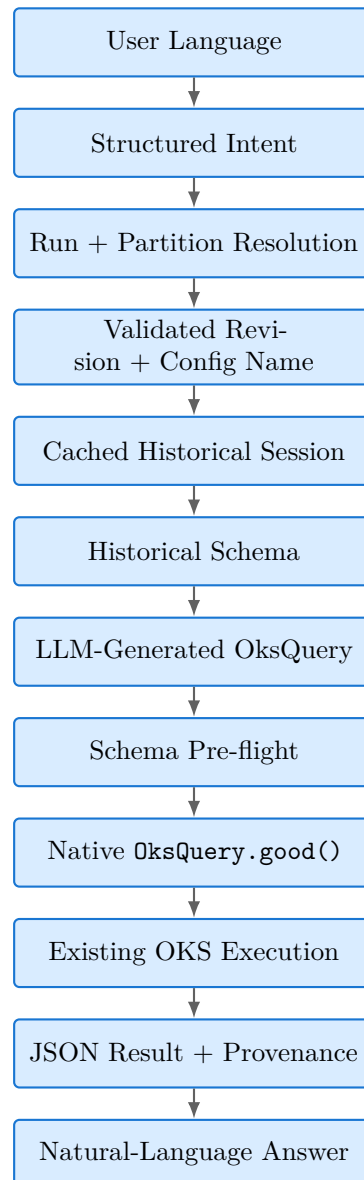
The final architecture follows one central principle:

The LLM adds semantic intelligence around the existing TDAQ/OKS configuration system; it does not replace the TDAQ/OKS configuration engine.

The LLM is responsible for understanding the user's language and constructing a candidate query. The existing TDAQ/OKS infrastructure remains responsible for:

- historical configuration access;
- repository resolution;
- schema representation;
- query validation;
- query execution;
- configuration object representation.

Therefore, the trust chain for a query is:



This architecture minimizes duplicated infrastructure, keeps historical configuration resolution deterministic, and ensures that generated queries are ultimately validated and executed by the native OKS system.